

# Clothes Segmentation

CVCY002 > 9-class semantic segmentation model (U-Net++), trained from scratch

<https://github.com/minanasser54/clothes-segmentation>

---

## 1 Dataset and Preprocessing

### 1.1 Dataset

People Clothing Segmentation dataset on Kaggle.

- **Volume & Resolution:** 1,000 paired image-mask samples at 825x550 resolution covering 59 original classes.
- **Selection Reason:** Preferred over alternatives (iMaterialist, Clothing Co-Parsing, ModaNet, DeepFashion) for providing ready-to-use PNG image/mask pairs with no format conversion needed.
- **Dataset Exploration:** .



### 1.2 Preprocessing & Augmentation Pipeline

- **Data Splits:** Matched by filename ID and split 80% train / 10% validation / 10% test (`data_preprocessing.py`).
- **Resizing:** Standardized to 256x256 pixels images via bilinear interpolation, masks via nearest-neighbor to avoid generating invalid intermediate class IDs.
- **Augmentation Strategy:** Applied exclusively to training data (`data_augmentation.py`):
  - **Joint Operations:** Horizontal flip and random crop-and-resize applied identically to both image and mask (using nearest-neighbor for masks).
  - **Image-Only Operations:** Brightness, contrast, and saturation jitter applied exclusively to images.
  - **Validation & Test Sets:** Left unaltered to evaluate real-world performance.

### 1.3 Class Remapping (59 -> 9 Superclasses)

- **Motivation:** Baseline model trained on 59 classes scored low mIoU (0.143), with over 20 classes stuck at 0.0 IoU (ring, watch, tie, etc.) due to severe data sparsity ( 1,600 augmented images across 59 classes).
- **Solution:** `class_mapping.py` consolidates 59 original classes into 9 semantic superclasses.
- **Execution:** Remapping is applied directly to full-resolution masks prior to resizing so the lookup table only sees valid original class IDs.

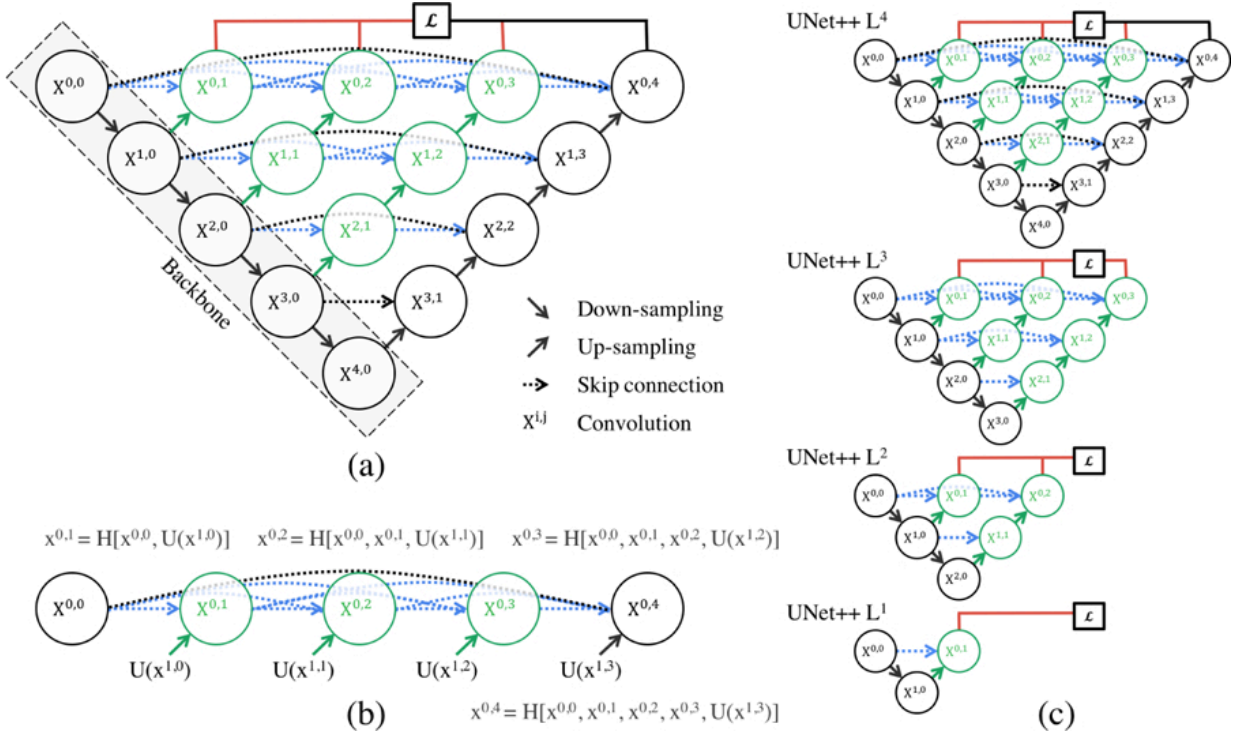
| Superclass    | Original classes folded in                                                                                                     |
|---------------|--------------------------------------------------------------------------------------------------------------------------------|
| background    | null                                                                                                                           |
| upper_clothes | blazer, blouse, bra, cape, cardigan, coat, hoodie, intimate, jacket, jumper, shirt, sweater, sweat-shirt, t-shirt, top, vest   |
| lower_clothes | jeans, leggings, panties, pants, shorts, skirt, stockings, tights                                                              |
| dress         | bodysuit, dress, romper, swimwear                                                                                              |
| shoes         | boots, clogs, flats, heels, loafers, pumps, sandals, shoes, sneakers, socks, wedges                                            |
| accessories   | accessories, bag, belt, bracelet, earrings, glasses, gloves, hat, necklace, purse, ring, scarf, sunglasses, tie, wallet, watch |
| skin          | skin                                                                                                                           |
| hair          | hair                                                                                                                           |
| suit          | suit                                                                                                                           |

## 2 Model Architecture

### 2.1 Overview

- **Architectures Implemented:** Standard U-Net baseline and U-Net++ (Zhou et al., 2018), both built from scratch in `model.py`. U-Net++ was selected for all final reported results.
- **Core Mechanism:** Replaces standard single skip connections with a grid of nested, densely connected convolution blocks. At encoder depth level  $i$  and nesting position  $j$ :

$$X_{i,j} = \text{Conv}(\text{Concat}(X_{i,0}, X_{i,1}, \dots, X_{i,j-1}, \text{Upsample}(X_{i+1,j-1})))$$



## 2.2 Design Choices

- **Bridging the Semantic Gap:** The decoder at each resolution receives a dense concatenation of all previously computed feature maps at that level. This progressively closes the gap between shallow (fine-detail) and deep (coarse-semantic) features prior to fusion.
- **Boundary Precision:** Tailored specifically for clothing segmentation challenges, such as recovering sleeve edges, collar lines, and borders between adjacent garments.
- **Omission of Deep Supervision:** Only the final, most-nested output node generates predictions without auxiliary losses at intermediate stages. This was a deliberate design choice to simplify implementation and debugging.
- **Capacity Tuning:** Uses feature widths [32, 64, 128, 256] across four levels (reduced from the paper’s 5-level default) to constrain parameter count for the 2400 pairs-image training dataset.

## 3 Loss Function

### 3.1 Loss Formulation

- **Hybrid Objective:** Combines region-based and distribution-based objectives:

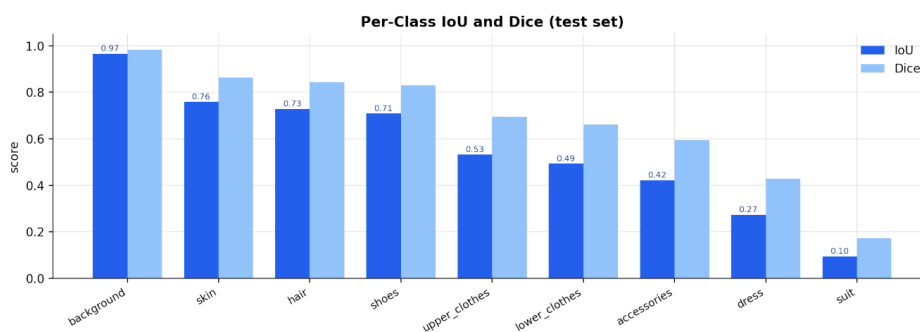
$$\text{total loss} = 0.7 \times \text{Dice loss} + 0.3 \times \text{weighted CE loss}$$

- **Dice Loss Role:** Directly optimizes region overlap (closely tied to IoU), mitigating severe spatial class imbalance (e.g., small clothing accessories vs. large background regions).
- **Weighted Cross-Entropy Role:** Weighted by inverse log-frequency per class to provide stable early-training gradients when predictions are near-random, avoiding initial Dice gradient instability.

## 4 Performance Analysis and Evaluation Metrics

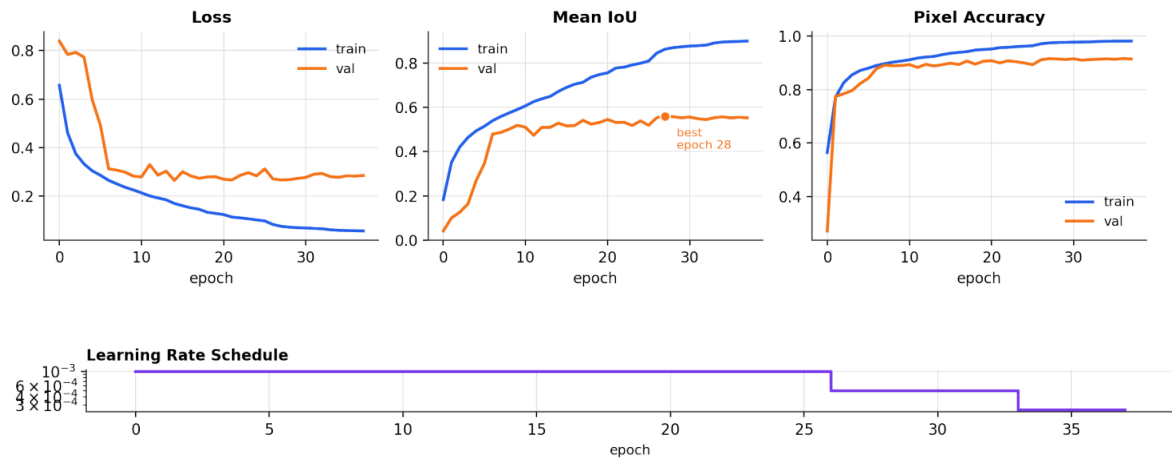
### 4.1 Metric Summary & Test Benchmarks

- **Primary Metrics:** Held-out test-set (100-pair split) evaluated in `eval.py`:
  - **Overall mIoU:** 0.553 (Macro-averaged IoU across all active classes).
  - **Foreground mIoU:** 0.502 (Excludes background to prevent metric inflation from large, easy-to-segment regions).
  - **Mean Dice Score:** 0.675 (Macro-averaged per-class Dice).
  - **Pixel Accuracy:** 90.7%.
- **Per-Class Breakdown:** Detailed IoU and Dice distributions across classes are captured below:



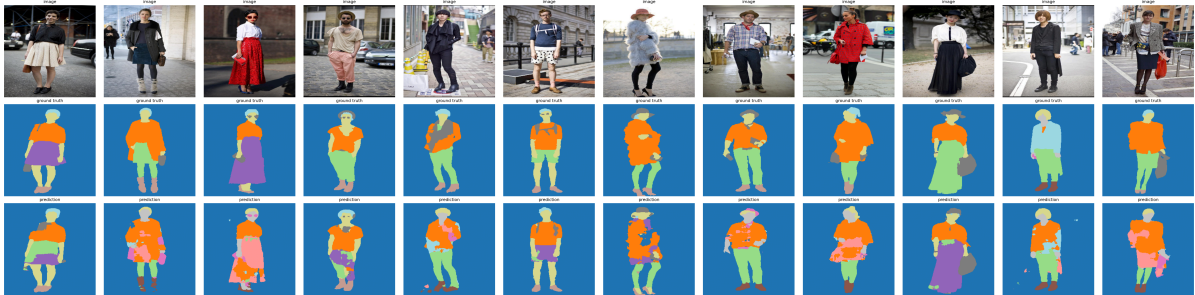
### 4.2 Training Dynamics & Checkpointing

- **Epochs Trained:** Completed 38 of a 60-epoch budget.
- **Early Stopping & Selection:** Both tracked `val_mean_iou`. The best checkpoint occurred at **Epoch 28** (`val_mean_iou`: 0.560, `val_accuracy`: 0.910), which was selected over the final epoch via `ModelCheckpoint(save_best_only=True)`.
- **Overfitting Gap:** Final-epoch `train_mean_iou` reached 0.899 vs. `val_mean_iou` of 0.552–0.560 (final train/val loss: 0.055 / 0.284). Retaining the epoch-28 weights protected performance against severe later-epoch overfitting.

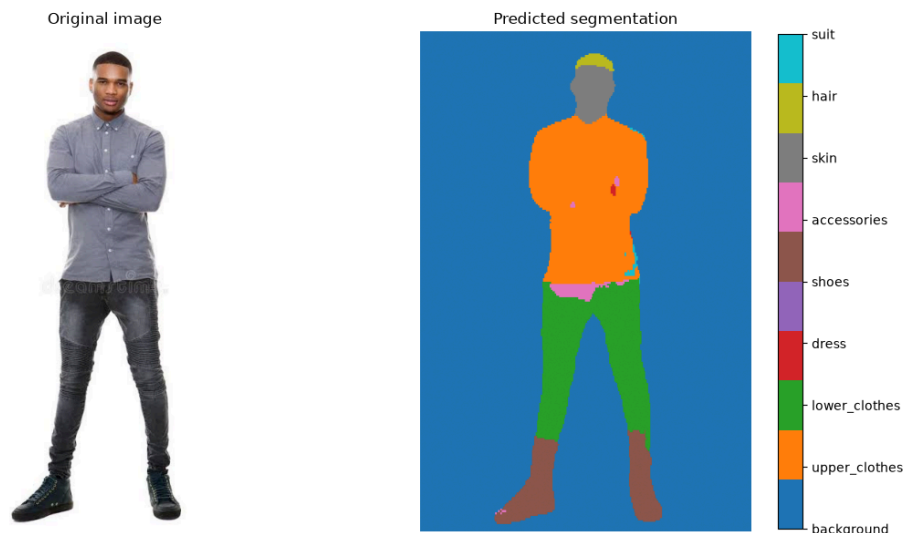


### 4.3 Qualities & Qualitative Evaluation

- **Visual Outputs:** Output samples demonstrate strong capture of general body silhouette and pose.



- **Strengths:** High contrast and broad region segmentation (skin, hair, upper/lower clothing) yield strong IoU. Demonstrates robustness to pose and background variations due to color and geometric augmentations, running with zero external pretrained weight dependencies.
- **Out of dataset Sample:**



## 5 Limitations of the System

### 5.1 Architectural & Qualitative Weaknesses

- **Prediction Noise:** Predictions exhibit internal region noise (speckled misclassifications within garments) and softer, less precise boundaries compared to ground truth.
- **Granularity Trade-off:** Superclass grouping (59 -> 9) prevents fine-grained garment distinction (e.g., t-shirts and sweaters are both categorized as `upper_clothes`).
- **Omitted Deep Supervision:** Omitting intermediate decoder losses simplified implementation but sacrificed potential boundary recovery features native to full U-Net++.
- **Low Resolution Constraints:** Resizing to fixed 256x256 inputs discards fine features such as thin straps, accessories, and garment textures to satisfy GPU memory constraints.

### 5.2 Training & Capacity Constraints

- **Scratch-Trained Capacity Limits:** Small dataset ( 2,400 augmented samples) combined with learning from scratch without pretrained encoders creates an unavoidable capacity bottleneck and causes early overfitting.

### 5.3 Operational & Domain Limitations

- **Single-Person Assumption:** Multi-person scenes fall outside training assumptions and produce unreliable masks.
- **Pose & Lighting Sensitivity:** Specialized fashion photography bias means extreme poses, heavy occlusions, unusual angles, low light, or backlit conditions are untested.
- **Domain Bias:** Optimized exclusively for real photographs; performance on artistic renders, sketches, or heavily stylized imagery is undefined.

---

Reproduce these numbers with: `uv run scripts/eval.py --run-name <your-run-name>`, which produces `models/<run-name>/eval/metrics.json` and `sample_predictions.png`.